

UNIT-I

Syllabus

- Linux Utilities
- File handling utilities,
- Security by file permissions,
- Process utilities,
- Disk utilities,
- Networking commands,
- Filters, Text processing utilities and Backup utilities,
- sed – scripts, operation, addresses, commands,
- applications, awk – execution, fields and records, scripts, operation,
- patterns, using system commands in awk.

Introduction

History of UNIX:

- UNIX is an Operating System (OS).
- Developed in 1969 at AT&T Bell Labs by Ken Thompson and Dennis Ritchie.
- Originally designed as a command-line, time-sharing system for mini-computers.
- Predecessor of Linux and many modern OS variants.

History of LINUX:

- LINUX was created by Linus Torvalds in 1991.
- It is an open-source operating system.
- Derived from UNIX, and shares many of its core principles and architecture.

Why LINUX/UNIX?

- ❖ Free and Open Source
- ❖ Customizable
- ❖ Highly Stable
- ❖ Secure Architecture
- ❖ Portable
- ❖ Written in C

File handling Utilities :

File handling commands in UNIX can be categorized into the following types:

File Creation and Deletion

1. Create a File : **Command:** cat > filename

Example: \$ cat > file1

Description : Creates a new file file1 and allows text entry.

2. Delete a File : **Command:** rm filename

Example: \$ rm file1

File Naming or Renaming

1. **Command :** mv oldname newname

Example : \$ mv file1 newfile

Editing Files : Use insert mode to add or modify content.

Editor Used : vi (Visual Editor) : **Command :** vi filename

Example : \$ vi f1.txt

Press Esc, then type :wq to save and exit.

View Edited Content: \$ cat f1.txt

File access permissions :

Unix operating system supports three types of users. They are,

User Types: 1. Owner 2. Group 3. Others

Permission Types: r – Read, w – Write, x – Execute

Change Permissions:

Command: chmod [permissions] filename

Example: \$ chmod a+rwX newfile

- Give execute permission to the user only: \$ chmod u+x script.sh
- Remove write permission from group: \$ chmod g-w report.txt
- Add read and execute permissions to all users: \$ chmod a+rx myfile
- Remove all permissions from others: \$ chmod o-rwx confidential.txt
- Set permissions: user = read/write, group = read, others = none
\$ chmod u=rw,g=r,o= file1

Numeric Mode Examples:

Permission	Binary	Octal
rwX	111	7
rw-	110	6
r--	100	4
r-x	101	5
--x	001	1

Examples:

- `$ chmod 744 program.c` // user: rwX (7), group: r-- (4), others: r-- (4)
- `$ chmod 666 data.txt` // Allow read/write for all users
- `$ chmod 755 run.sh` // Make a file executable by everyone
- `$ chmod 700 private.sh` // Give only the owner read, write, and execute permissions:

Commands

ln COMMAND:

ln command is used to create link to a file (or) directory.

SYNTAX: ln [options] existingfile(or directory)name newfile(or directory)name

OPTIONS:

1 symbolic link

2. Interactive link

3. Force link

Symbolic link:

Created using the -s option.

Acts like a shortcut to the original file.

If you open the link, you are actually opening the original file.

If the original file is deleted, the link is broken.

Commands

Syntax:

```
$ ln -s [original_file_or_dir] [link_name]
```

Example:

```
$ ln -s file1.txt mylink.txt
```

2. Interactive Link :

Asks for confirmation before replacing an existing link or file.

Example:

```
ln -i file1.txt file2.txt
```

3. Force Link :

Forces the creation of the link without asking, even if the target file already exists.

Example: `ln -sf file1.txt file2.txt`

Commands

mv COMMAND:

mv command which is short for move. It is used to move/rename file from one directory to another. mv command is different from cp command as it completely removes the file from the source and moves to the directory specified, where cp command just copies the content from one file to another.

SYNTAX: `$mv [-f] [-i] oldname newname`

OPTIONS:

-f This will not prompt before overwriting (equivalent to --reply=yes). mv
-i will move the file(s) without prompting even if it is writing over an existing target.

Commands

EXAMPLE:

1. To Rename / Move a file:

```
mv file1.txt file2.txt
```

This command renames file1.txt as file2.txt

2. To move a directory

```
mv hscripts tmp
```

In the above line mv command moves all the files, directories and sub-directories from hscripts folder/directory to tmp directory if the tmp directory already exists. If there is no tmp directory it rename's the hscripts directory as tmp directory.

3. To Move multiple files/More files into another directory

```
mv file1.txt tmp/file2.txt newdir
```

This command moves the files file1.txt from the current directory and file2.txt from the tmp folder/directory to newdir.

Commands

rm COMMAND:

rm linux command is used to remove/delete the file from the directory.

SYNTAX: `$rm [options..] [file | directory]`

OPTIONS:

- f Remove all files in a directory without prompting the user.
- I Interactive. With this option, rm prompts for confirmation before removing any files.
- r (or) -R Recursively remove directories and subdirectories in the argument list. The directory will be emptied of files and removed. The user is normally prompted for removal of any write-protected files which the directory contains.

EXAMPLE: 1) To Remove / Delete a file:

```
rm file1.txt
```

Here rm command will remove/delete the file file1.txt.

Commands

2. To delete a directory tree:

```
rm -ir tmp
```

This rm command recursively removes the contents of all subdirectories of the tmp directory, prompting you regarding the removal of each file, and then removes the tmp directory itself.

3. To remove more files at once

```
rm file1.txt file2.txt
```

rm command removes file1.txt and file2.txt files at the same time.

Commands

mkdir COMMAND:

mkdir command is used to create one or more directories.

SYNTAX: mkdir [options] directories

OPTIONS:

- m Set the access mode for the new directories.
- p Create intervening parent directories if they don't exist.
- v Print help message for each directory created.

EXAMPLE:

1. Create directory:

```
mkdir test
```

The above command is used to create the directory 'test'.

2. Create directory and set permissions:

```
mkdir -m 666 test
```

The above command is used to create the directory 'test' and set the read and write permission.

Commands

rmmdir COMMAND:

rmmdir command is used to delete/remove a directory and its subdirectories.

SYNTAX: The Syntax is rmmdir [options..] Directory

OPTIONS: -p Allow users to remove the directory dirname and its parent directories which become empty.

EXAMPLE:

1. To delete/remove a directory

rmmdir tmp

rmmdir command will remove/delete the directory tmp if the directory is empty.

2. To delete a directory tree:

rm -ir tmp

This command recursively removes the contents of all subdirectories of the tmp directory, prompting you regarding the removal of each file, and then removes the tmp directory itself.

Commands

cp COMMAND:

cp command copy files from one location to another. If the destination is an existing file, then the file is overwritten; if the destination is an existing directory, the file is copied into the directory (the directory is not overwritten).

SYNTAX: `$cp [OPTIONS]... SOURCE DEST`

`cp [OPTIONS]... SOURCE... DIRECTORY`

`cp [OPTIONS]... --target-directory=DIRECTORY SOURCE...`

OPTIONS:

`--a` same as `-dpR`.

`--backup[=CONTROL]` make a backup of each existing destination file

`-b` like `--backup` but does not accept an argument.

`-f` if an existing destination file cannot be opened, remove it and try

Commands

Example4: Search for a file name test.txt and its permissions are 775 in a given box

```
find / -perm 775 -name test.txt
```

Example5: How about searching files with SUID bit set and file permissions are 755?

```
find / -perm 4755
```

Example6: Search for all the files with name test.txt and the owner of this file is amar

```
find / -user Surendra -name test.txt
```

Example7: find all the files whos name is test.txt and owned by a group called redcluster

```
find / -group redcluster -name test.txt
```

Commands

Pwd COMMAND:

Short for print working directory the pwd command displays the name of the current working directory.

Syntax

pwd

Examples

pwd

Typing pwd at the prompt would give you something similar to:

/home/computerhope/public_html

Security by file permissions

File ownership is an important component of UNIX that provides a secure method for storing files. Every file in UNIX has the following attributes:

Owner permissions: The owner's permissions determine what actions the owner of the file can perform on the file.

Group permissions: The group's permissions determine what actions a user, who is a member of the group that a file belongs to, can perform on the file.

Other (world) permissions: The permissions for others indicate what action all other users can perform on the file.

The Permission Indicators:

While using `ls -l` command it displays various information related to file permission as follows:

```
$ls -l -rwxr-xr-- 1 root  users 1024 Nov 2 00:10 myfile
drwxr-xr--- 1 root  users 1024 Nov 2 00:10 mydir
```

Here first column represents different access mode ie. permission associated with a file or directory.

Security by file permissions

File Access Modes:

The permissions of a file are the first line of defense in the security of a Unix system. The basic building blocks of Unix permissions are the **read**, **write**, and **execute** permissions, which are described below:

Read: Grants the capability to read ie. view the contents of the file.

Write: Grants the capability to modify, or remove the content of the file.

Execute: User with execute permissions can run a file as a program

Changing Permissions:

To change file or directory permissions, you use the **chmod** (change mode) command. There are two ways to use chmod: symbolic mode and absolute mode.

Using chmod in Symbolic Mode:

The easiest way for a beginner to modify file or directory permissions is to use the symbolic mode. With symbolic permissions you can add, delete, or specify the permission set you want by using the operators in the following table.

Security by file permissions

Chmod operator	Description
+	Adds the designated permission(s) to a file or directory.
-	Removes the designated permission(s) from a file or directory.
=	Sets the designated permission(s).

Here's an example using testfile. Running `ls -l` on testfile shows that the file's permissions are as follows:

\$ls -l testfile-rwxrwxr-- 1 amrood users 1024 Nov 2 00:10 testfile Then each example chmod command from the preceding table is run on testfile, followed by `ls -l` so you can see the permission changes:

Security by file permissions

```
$chmod o+wx testfile
```

```
$ls -l testfile
```

```
-rwxrwxrwx 1 root  users 1024  Nov 2 00:10  testfile
```

```
$chmod u-x testfile
```

```
$ls -l testfile
```

```
-rw-rwxrwx 1 root  users 1024  Nov 2 00:10  testfile
```

```
$chmod g=r-x testfile
```

```
$ls -l testfile
```

```
-rw-r-xrwx 1 root  users 1024  Nov 2 00:10  testfile
```

Security by file permissions

Here's how you could combine these commands on a single line:

```
$chmod o+wx,u-x,g=r-x testfile
```

```
$ls -l testfile
```

```
-rw-r-xrwx 1 root  users 1024 Nov 2 00:10 testfile
```

Using chmod with Absolute Permissions:

The second way to modify permissions with the chmod command is to use a number to specify each set of permissions for the file.

Each permission is assigned a value, as the following table shows, and the total of each set of permissions provides a number for that set.

Security by file permissions

Number	Octal Permission Representation	Ref
0	No permission	---
1	Execute permission	--X
2	Write permission	-W-
3	Execute and write permission: 1 (execute) + 2 (write) = 3	-wX
4	Read permission	r--
5	Read and execute permission: 4 (read) + 1 (execute) = 5	r-X
6	Read and write permission: 4 (read) + 2 (write) = 6	rw-
7	All permissions: 4 (read) + 2 (write) + 1 (execute) = 7	rwX

Security by file permissions

Here's an example using testfile. Running `ls -l` on testfile shows that the file's permissions are as follows:

```
$ls -l testfile
```

```
-rwxrwxr-- 1 root  users 1024  Nov 2 00:10  testfile
```

Then each example `chmod` command from the preceding table is run on testfile, followed by `ls -l` so you can see the permission changes:

```
$ chmod 755 testfile
```

```
$ls -l testfile
```

```
-rwxr-xr-x 1 root  users 1024  Nov 2 00:10  testfile
```

```
$chmod 743 testfile
```

```
$ls -l testfile
```

```
-rwxr---wx 1 root  users 1024  Nov 2 00:10  testfile
```

```
$chmod 043 testfile
```

Security by file permissions

Changing Owners and Groups:

While creating an account on Unix, it assigns a owner ID and a group ID to each user. All the permissions mentioned above are also assigned based on Owner and Groups.

Two commands are available to change the owner and the group of files:

chown: The chown command stands for "change owner" and is used to change the owner of a file.

chgrp: The chgrp command stands for "change group" and is used to change the group of file.

Changing Ownership:

The chown command changes the ownership of a file. The basic syntax is as follows:

```
$ chown [new_owner] [file_name]
```

The value of user can be either the name of a user on the system or the user id (uid) of a user on the system.

Following example:

```
$ chown root1 testfile
```

Changes the owner of the given file to the user **root1**

NOTE: The super user, root, has the unrestricted capability to change the ownership of a any file but normal users can change only the owner of files they own.

Security by file permissions

Changing Group Ownership:

- The chgrp command changes the group ownership of a file. The basic syntax is as follows:

 \$ chgrp [new_group] [file_name]
- The value of group can be the name of a group on the system or the group ID (GID) of a group on the system.
- Following example: \$ chgrp special testfile
- Changes the group of the given file to **special** group.

Process utilities

ps COMMAND:

ps command is used to report the process status. ps is the short name for Process Status.

SYNTAX: The Syntax is ps [options]

OPTIONS:

- e List information about every process now running.
- f Generates a full listing.
- u option followed by user-id displays the processes owned by the user-id.
- l Generate a long listing.

Process utilities

EXAMPLE:

1. ps

Output:

```
PID TTY TIME CMD
```

```
2540 pts/1 00:00:00 bash
```

```
2621 pts/1 00:00:00 ps
```

In the above example, typing ps alone would list the current running processes.

2. ps -f

Output:

UID	PID	PPID	C	STIME	TTY	TIME	CMD
abc	2540	2536	0	15:31	pts/1	00:00:00	bash
xyz	2639	2540	0	15:51	pts/1	00:00:00	

ps -f

Displays full information about currently running processes.

Process utilities

who COMMAND:

who command can list the names of users currently logged in, their terminal, the time they have been logged in, and the name of the host from which they have logged in.

SYNTAX: who [options] [file]

OPTIONS:

-H Print column headings above the output.

-u provides a detailed list

EXAMPLE:

1. who -uH

Output:

NAME	LINE	TIME	IDLE	PID	COMMENT
hiox	ttyp3	Jul 10 11:08	.	4578	

This sample output was produced at 11 a.m. The "." indicates activity within the last minute.

Process utilities

2. \$who am i

who am i command prints the user name.

w command:

w - show who is logged on and what they are doing

DESCRIPTION

w displays information about the users currently on the machine, and their processes. The header shows, in this order, the current time, how long the system has been running, how many users are currently logged on, and the system load averages for the past 1, 5, and 15 minutes.

Kill COMMAND:

Kill command in UNIX and Linux is normally used to kill a suspended or hanged process or process group.

Example:

“if you issue the ps command and find that one of your processes is hung or if you started a large job that you wish to stop, you can do so with the kill command”

Process utilities

The following example shows how to kill a process,

```
$ps -ef | grep sales
```

```
Sales 19336 19334 0 05:24:32 pts/4 0:01 -ksh
```

```
Sales 19426 19336 0 06:01:01 pts/4 0:00 ora
```

Kill 19426 This will kill the process ora with PID 19426

To terminate more than one process at a time, use the following syntax

```
$kill signal PID PID PID
```

```
$pkill signal process process process
```

In kill utility we will give PID where as in pkill utility we have to give the name of the process

kill -9 is used to forcefully terminate a process in Unix. Here is syntax of kill command in UNIX.

```
ps -ef| grep process_identifier // will give you PID
```

```
kill -9 PID
```


Disk utilities

The utilities which will display the information about the disk is known as the disk utilities.

The following mentioned are the disk utilities which are used in common,

- 1) df
- 2) du

df COMMAND:

df command is used to find the empty disk space. Df stands for disk free.

Example:

```
$ df
```

Filesystem	1K-blocks	Used	Available	Use%	Mounted on
/dev/sda1	132239776	6210884	119311504	5%	/
/tmpfs	4021876	0	4021876	0%	/dev/shm
/dev/sdb2	30969600	117740	29278696	1%	/home/oracle
/dev/sdc1	576310180	71232	546964104	1%	/home/data

Du COMMAND:

The du command displays the disk space used by the specified files and directories.

Disk utilities

Syntax:

`du [option]....[file]`

Options:

- 1) `-a`: it writes counts for all files, not just for directories .
- 2) `-b`: it prints the size in bytes.
- 3) `-c`: it prints the grand total (i.e, the total disk space used by the directory).
- 4) `-h`: it prints the human readable formats.example1k,2g
- 5) `-s`: it does not show the size of sub-directories.

```
[amar@localhost ~]$ du -a
```

```
8    ./zshrc
```

```
8    ./f1
```

```
8    ./bashrc
```

```
8    ./kde/Autostart/.directory
```

```
16   ./kde/Autostart
```

```
24   ./kdehst
```

Networking commands

The commands which we will use for the communication between more than one system for sharing resources or any other issue are known as the networking commands.

Commands such as telnet, ftp, rlogin, finger are commonly used networking commands.

telnet COMMAND:

telnet is a internet protocol which enables user to connect a remote system and transfer data.

Syntax: \$ telnet hostname or telnet ip_address

Example : *\$ telnet itctnss0123*

Trying 164.130....

Connected to itctnss0123.

Escape character is '^]'.

SunOS 5.8

login: root

Password:

Last login: Thu Feb 24 16:13:12 from itctnss0123

Sun Microsystems Inc. SunOS 5.8 Generic Patch February 2004

Networking commands

Finger COMMAND:

The finger command is used to find out the details of a user, on both local and remote system.

Finger may be disabled on other systems for security reasons.

Following are the simple syntax to use finger command:

Check all the logged in users on local machine as follows:

Examples:

```
[amar@localhost ~]$ finger
```

Login	Name	Tty	Idle	Login Time	Office	Office Phone
amar		pts/1		May 25 10:06	(172.16.10.5)	
root	root	pts/0	7d	May 16 14:03	(:0.0)	

```
[amar@localhost ~]$ finger amar
```

```
Login: amar                      Name: (null)
Directory: /home/amar            Shell: /bin/bash
On since Sat May 25 10:06 (IST) on pts/1 from 172.16.10.5
No mail.
No Plan.
```

Networking commands

Ftp is the user interface to the **Internet** standard File Transfer Protocol. The program allows a user to transfer files to and from a remote network site. This utility helps you to upload and download your file from one computer to another computer.

The ftp utility has its own set of UNIX like commands which allow you to perform tasks such as:

- Connect and login to a remote host.

- Navigate directories.

- List directory contents

- Put and get files

- Transfer files as ascii, ebcdic or binary

Syntax:

Following is the simple syntax to use **ping** command:

```
$ftp hostname or ip-address
```

Networking commands

```
[amar@localhost ~]$ ftp amar
ftp: amar: unknown host
ftp> quit
[amar@localhost ~]$ ftp 172.16.1.254
ftp: connect: Connection refused
ftp>
ftp> quit
```

Command	Description
put filename	Upload filename from local machine to remote machine.
get filename	Download filename from remote machine to local machine.
mput file list	Upload more than one files from local machine to remote machine.
mget file list	Download more than one files from remote machine to local machine.
prompt off	Turns prompt off, by default you would be prompted to upload or download movies using mput or mget commands.
prompt on	Turns prompt on.
dir	List all the files available in the current directory of remote machine.
cd dirname	Change directory to dirname on remote machine.
lcd dirname	Change directory to dirname on local machine.
quit	Logout from the current login.

Networking commands

rlogin COMMAND:

rlogin command helps the user to log on to his identical remote account, without using either the username or password.

The “*rlogin*” command is similar to telnet. The rlogin starts a terminal session on a remote host.

Syntax : *\$rlogin hostname*

Example : 1) [amar@localhost ~]\$ rlogin 172.16.1.254

connect to address 172.16.1.254 port 543: Connection refused

Trying krb4 rlogin...

connect to address 172.16.1.254 port 543: Connection refused

trying normal rlogin (/usr/bin/rlogin)

2) [amar@localhost ~]\$ rlogin amar

amar: host unknown

Trying krb4 rlogin...

amar: host unknown

trying normal rlogin (/usr/bin/rlogin)

amar: Unknown host

Filters

a filter is a program that accepts input, transforms it and outputs the transformed data. The idea of the filter is closely associated with several ideas that are part of the UNIX operating system: standard input and output, input/output redirection and pipes.

Standard input and output refer to default locations from which a program will take input and to which it will write output. The standard input (STDIN) for a program running interactively at the command line is the keyboard; the standard output (STDOUT) is the terminal screen.

With input/output redirection, a program can take input or send output using a location other than standard input or output—a file, for example. Redirection of STDIN is accomplished using the < symbol, redirection of STDOUT by >.

Filters

cat COMMAND:

cat linux command concatenates files and print it on the standard output.

SYNTAX: \$ cat [OPTIONS] [FILE]...

OPTIONS:

- A Show all.
- b Omits line numbers for blank space in the output.
- e A \$ character will be printed at the end of each line prior to a new line.
- E Displays a \$ (dollar sign) at the end of each line.
- n Line numbers for all the output lines.
- s If the output has multiple empty lines it replaces it with one empty line.
- T Displays the tab characters in the output.
- v Non-printing characters (with the exception of tabs, new-lines and form-feeds) are printed visibly.

Examples:

1. To Create a new file:

cat > file1.txt

This command creates a new file file1.txt. After typing into the file press control+d (^d) simultaneously to end the file.

Filters

2. To Append data into the file:

```
cat >> file1.txt
```

To append data into the same file use append operator >> to write into the file, else the file will be overwritten (i.e., all of its contents will be erased).

3. To display a file:

```
cat file1.txt
```

This command displays the data in the file.

4. To concatenate several files and display:

```
cat file1.txt file2.txt
```

The above cat command will concatenate the two files (file1.txt and file2.txt) and it will display the output in the screen. Some times the output may not fit the monitor screen. In such situation you can print those files in a new file or display the file using less command.

```
cat file1.txt file2.txt | less
```

Filters

5. To concatenate several files and to transfer the output to another file.

```
cat file1.txt file2.txt > file3.txt
```

In the above example the output is redirected to new file file3.txt. The cat command will create new file file3.txt and store the concatenated output into file3.txt.

Display beginning and end of files

head COMMAND:

head command is used to display the first ten lines of a file, and also specifies how many lines to display

SYNTAX: \$ head [options] filename

Filters

OPTIONS:

- n To specify how many lines you want to display.
- n number The number option-argument must be a decimal integer whose sign affects the location in the file, measured in lines.
- c number The number option-argument must be a decimal integer whose sign affects the location in the file, measured in bytes.

EXAMPLE:

1. head f1.txt

This command prints the first 10 lines of ' f1.txt '.

2. head -5 f1.txt

The head command displays the first 5 lines of ' f1.txt '.

3. head -c 5 f1.txt

The above command displays the first 5 characters of ' f1.txt '.

tail COMMAND:

Tail command is used to display the last or bottom part of the file. By default it displays last 10 lines of a file.

SYNTAX:

\$ tail [options] filename

Filters

OPTIONS:

- l **To specify the units of lines.**
- b To specify the units of blocks.
- n To specify how many lines you want to display.
- c number The number option-argument must be a decimal integer whose sign affects the location in the file, measured in bytes.
- n number The number option-argument must be a decimal integer whose sign affects the location in the file, measured in lines.

EXAMPLE:

1. tail index.php
It displays the last 10 lines of 'index.php'.
2. tail -2 index.php
It displays the last 2 lines of 'index.php'.
3. tail -n 5 index.php
It displays the last 5 lines of 'index.php'.
4. tail -c 5 index.php
It displays the last 5 characters of 'index.php'.

Filters

cut COMMAND:

cut command is used to cut out selected fields of each line of a file. The cut command uses delimiters to determine where to split fields.

SYNTAX:

cut [options]

OPTIONS:

- c** Specifies character positions.
- b** Specifies byte positions.
- d** flags Specifies the delimiters and fields.

EXAMPLE:

1. cut -c1-3 text.txt

Output:

Thi
Cut the first three letters from the above line.

2. cut -d, -f1,2 text.txt

Output:

This is, an example program

The above command is used to split the fields using delimiter and cut the first two fields.

Filters

paste COMMAND:

paste command is used to paste the content from one file to another file. It is also used to set column format for each line.

SYNTAX:

\$ paste [options]

OPTIONS:

- s Paste one file at a time instead of in parallel.
- d Reuse characters from LIST instead of TABs .

EXAMPLE:

1. paste test.txt>test1.txt

Paste the content from 'test.txt' file to 'test1.txt' file.

2. ls | paste - - - -

List all files and directories in four columns for each line.

sort COMMAND:

sort command is used to sort the lines in a text file.

SYNTAX:

sort [options] filename

OPTIONS:

- r Sorts in reverse order.
- u If line is duplicated display only once.
- o filename Sends sorted output to a file.

Filters

EXAMPLE:

1. `sort test.txt`

Sorts the 'test.txt' file and prints result in the screen.

2. `sort -r test.txt`

Sorts the 'test.txt' file in reverse order and prints result in the screen

tr COMMAND:

- `tr` filter translates specified characters into other characters.

Normal usage

`tr string1 string2`

- Translates any character read from stdin that is in `string1` into the corresponding character from `string2`; otherwise the character is simply passed through.

- `string1` and `string2` normally have the same number of characters.

- Eg `tr abc X5Z`

translates a into X, b into 5 and c into Z.

Filters

uniq COMMAND:

- `uniq` assumes lines of ASCII file are sorted and deals with unique lines depending on the option given.
- No option, one copy of each unique line is printed to stdout. Duplicated lines are discarded.

- Eg `uniq dups`

dups: default output:

A	a
b	b
b	c
c	dd
dd	ddd
ddd	
ddd	
Ddd	

- `-u` option prints only those lines that have no duplicates.

- Eg `uniq -u dups`

dups: -u output:

a	a
b	c
b	dd
c	

Filters

-d option prints only those lines that have duplicates.

Eg `uniq -d dups`

dups: -d output:

a	b
b	ddd
b	
c	
dd	
ddd	
ddd	
ddd	

`uniq` runs as filter or as program driving a pipeline.

There are options to compare fields and to count duplicated lines.

Filters

Wc COMMAND:

The purpose of wc command is to count the number of lines, words, characters in a particular file or files

Syntax: `$ wc filename`

Example:

```
$ wc file1
```

```
5  40  200   file1
```

First column represents number of lines

Second column represents number of words

Third column represents number of characters

Fourth column shows the file name

OPTIONS:

`-l` (line)

`-w` (words)

`-c` (character)

Filters

cmp COMMAND:

Compares two files and tells you what line numbers are different.

By default location of first mismatch is displayed

Syntax: `$ cmp [-options] filename1 filename2`

- l this option displays the list of all differences present in the file byte-by-byte.
- s Write nothing for differing files; return exit status('0' or '1') only.
the '0' specifies that two files are similar and '1' specifies that there is at least one that is different.

Text processing utilities

- The operations performed by the text processing utilities are as follows
- Processing the text
- Modifying the content of files
- Deleting words and characters from files

cat: cat is used to create the files.

```
$cat> filename
```

Type some text here

Press ctrl+d

```
$
```

Cat can also be used to display the contents Of a file.

```
$cat filename
```

Cat can also concatenate the contents of 2 files and store them in third file.

```
Cat>file1 file2>new file
```

To append the contents of two files into another file use

Text processing utilities

tail:

tail command displays the end of the file. It displays the last ten lines by default.

```
$tail file
```

To display last 3 lines use

```
$tail -n 3 file      or
```

```
$tail -3 file
```

We can also address the lines from the beginning of the file instead of the end. The + count allows to do that.

Text processing utilities

head:

head command as the name implies, displays the top of the file. When used without an option, it displays the first 10 lines of the file.

```
$head file
```

We can use `-n` option to specify a line count and display, say first 3 lines of the file.

```
$head -n 3 file or $head -3 file
```

Sort:

Sort can be used for sorting the contents of a file.

```
$sort shortlist
```

Sorting starts with the first character of each line and proceeds to the next character only when the characters in two lines are identical.

Sort options:

With `-t` option sorts a file based on the fields.

Text processing utilities

```
$sort -t "|" +2 shortlist
```

The sort order can be reversed with `-r` option.

Sorting on secondary key:

U can sort on more than one field i.e. u can provide a secondary key to sort.

If the primary key is the third field and the secondary key the second field, we can use

```
$sort -t \| +2 -3 +1 shortlist
```

Numeric sort (`-n`):

To sort on number field use sort with `-n` option.

```
$sort -t: +2 -3 -n group1
```

Removing duplicate lines (`-u`):

The `-u` option u purge duplicate lines from a file.

Text processing utilities

nl:

nl is used for numbering lines of a file.

Nl numbers only logical lines –those containing something other apart from the new line character.

\$nl file

nl uses a tab as a default delimiter, but we can change it with -s option.

\$nl -s: file

nl won't number a line if it contains nothing.

Grep: globally search for a regular expression and print.

Grep scans a file for the occurrence of a pattern and depending on the options used, displays

Lines containing the selected pattern.

Lines not containing the selected pattern (-v).

Line numbers where pattern occurs (-n)

No. of lines containing the pattern (-c)

Text processing utilities

- File names where pattern occurs (-l)

Syntax:

grep option pattern filename(s)

Egrep: extended grep

Egrep extended set includes 2 special characters + and ?.

+ --matches one or more occurrences of the pervious character.

?-- matches zero or more occurrences of the pervious character.

- **fgrep:** fast grep
- If search criteria requires only sequence expressions, fgrep is the best utility.
- Fgrep supports only string patterns, no regular expressions.
- To extract all the lines that contain an apostrophe use fgrep as follows:
- \$fgrep "" file

Text processing utilities

- **Cut:** slitting the file vertically

U can slice a file vertically with cut command.

- Cutting columns(-c):

Cut with -c option cuts the columns.

To extract first 4 columns of the group file :

```
$cut -c 1-4 group1
```

The specification -c 1-4 cuts columns 1 to 4.

Cutting fields:

To cut 1st and 3rd fields use

```
$cut -d: -f1,3 group1
```

Text processing utilities

- **Paste:** pasting files
- What u cut with the cut can be pasted back with paste command-but vertically rather than horizontally. u can view two files side by side by pasting them.
- To join two files calc.lst and result.lst use
- `$paste -d= calc.lst result.lst`
- **Join:**
- is a command in Unix-like operating systems that merges the lines of two sorted text files based on the presence of a common field.
- The join command takes as input two text files and a number of options. If no command-line argument is given, this command looks for a pair of lines from the two files having the same first field (a sequence of characters that are different from space), and outputs a line composed of the first field followed by the rest of the two lines.
- `$join file1 file2`

Back up utilities

The two back up utilities are used in Linux. They are,

- 1) Cpio
- 2) Tar

cpio COMMAND: copy input-output

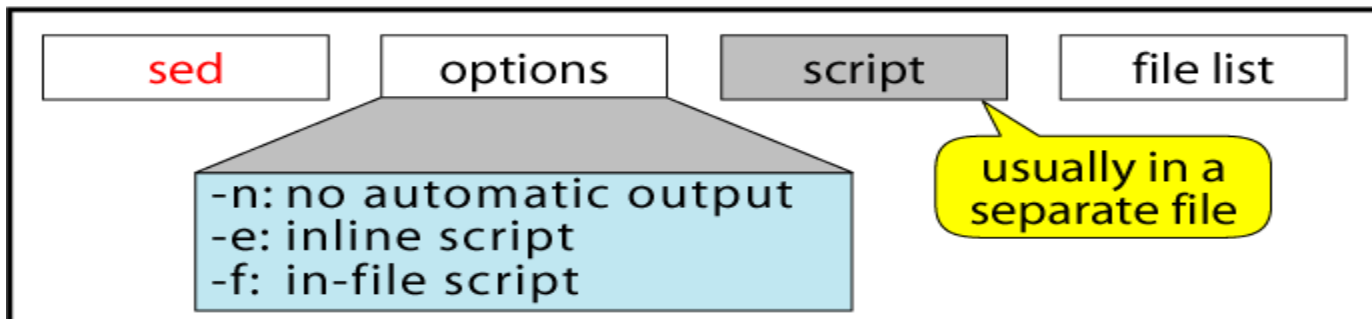
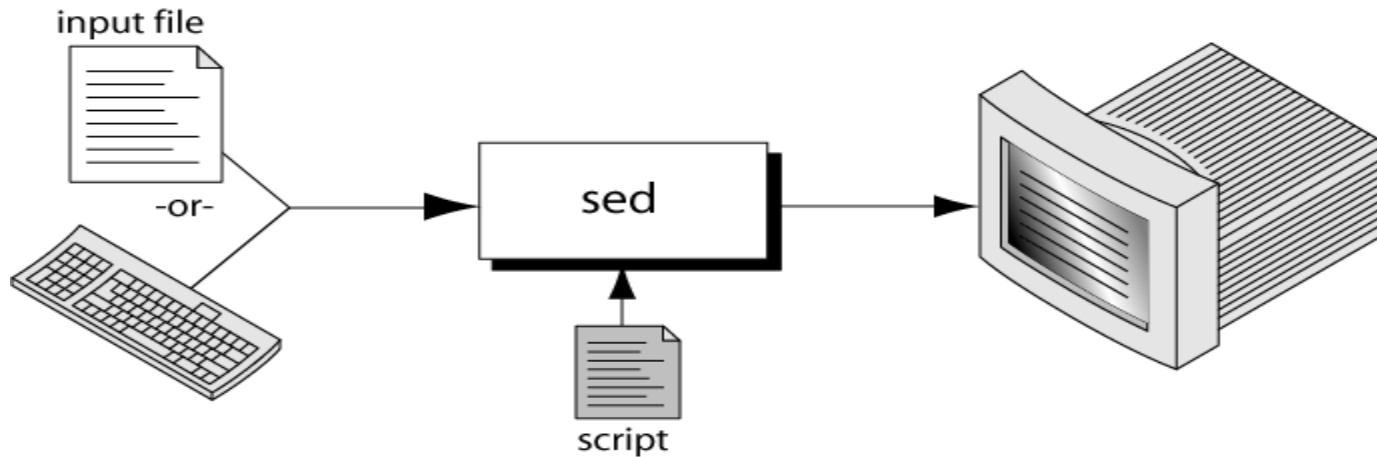
- Cpio command copies files to and from a backup device. It uses standard input to take the list of filenames.
- It then copies them with their contents and headers into stream which can be redirected to a file or a device.
- Cpio can be used with redirection and piping.
- Cpio uses two options -o (output) and -i (input) either of which must be there in the command line.

Back up utilities

- **tar COMMAND** : the tape archive program
- Tar doesn't normally write to the standard output but creates an archive in the media.
- Tar accepts file and directory names as arguments.
- It operates recursively.
- It can create several versions of same file in a single archive.
- It can append to an archive without overwriting the entire archive.
- -c option is used to copy files to backup device.
- `$tar -cvf /dev/rdisk/foq18dt /home/sales/sql/*.sql`
- The verbose option (-v) shows the no. of blocks used by each file.
- Files are restored with the -x (extract) key option. when no file or directory name is specified it restores all files from the backup device.

Sed

- Sed is one of the most powerful filters. it stands for stream editor. Despite its name editing, it does not modify the original file. It reads the standard input (a keyboard or file), processes it using a separate file called sed script and writes the standard output.



Sed

Options:

- e: it is a default option. It indicates that the script is on the command line.
- f: it indicates that script is in a file which immediately follows this option.
- n: it suppresses the automatic output. That is, it will not display the contents of the pattern space.

Scripts, Operation

A script is nothing more than a file of commands or instructions.

Each command consists of an address and an action, where the address can be a pattern (regular expression)

address	!	Action
---------	---	--------



Complement operator(optional)
A Sed instruction

```
$ sed -e 'address command' input_file
```



(a) Inline Script

```
$ sed -f script.sed input_file
```

(b) Script File

Scripts

Here is an example of sed script namely sample.sed

Sample.sed

```
# this is one line comment
```

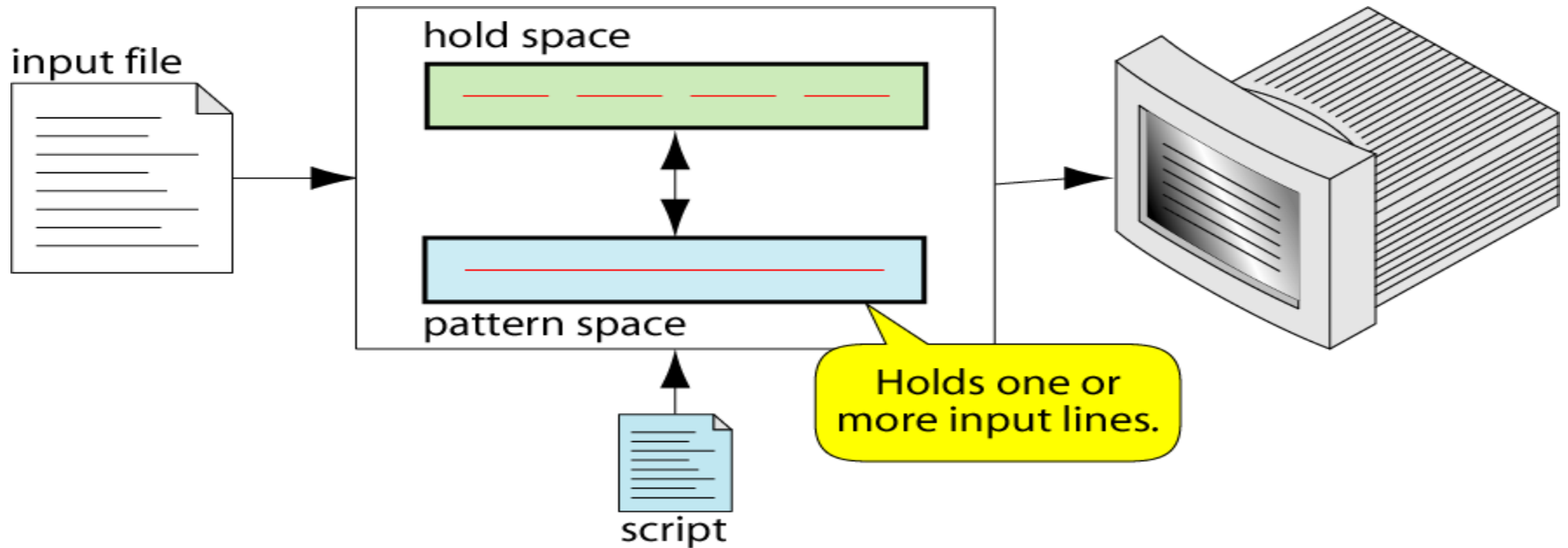
```
#this is another comment
```

```
3d
```

```
1,10s/friends/buddies
```

In the sample script, the first two line are the comments. The third and fourth lines are the instructions. The former specifies the third line in the input file is to be deleted while the later specifies to replace all occurrences of “friends” by “buddies ” in the input lines 1 through 10.

Scripts, Operation



As each line of the input file is read, *sed* reads the first command of the script and checks the address or pattern against the current input line

If there is a match, the command is executed

If there is no match, the command is ignored

Addresses

Address determines which lines in the input file are to be processed by the command(s)

if no address is specified, then the command is applied to each input line

Address types:

- Single-Line address
- Set-of-Lines address
- Range address
- Nested address

Single-Line Address

- Specifies only one line in the input file
- special: dollar sign (\$) denotes last line of input file

Addresses

Examples:

- show only line 3

sed -n -e '3 p' input-file

- show only last line

sed -n -e '\$ p' input-file

- substitute —"endif" with —"fi" on line 10

sed -e '10 s/endif/fi/' input-file

Set-of-Lines Address

- use regular expression to match lines
- written between two slashes
- process only lines that match
- may match several lines
- lines may or may not be consecutives

Examples:

- `/^A/command` (Matches all lines that start with 'A')
- `/^$/colmmand` (Matches blank lines)

Addresses

Range Address

- Defines a set of consecutive lines

Format:

- start-addr,end-addr (inclusive)

following are the possible combinations

- 10,50 line-number,line-number
- 10,/R.E/ line-number,/RegExp/
- /R.E./,10 / RegExp/,line-number
- /R.E./,/R.E/ /RegExp/,/RegExp/

Examples :

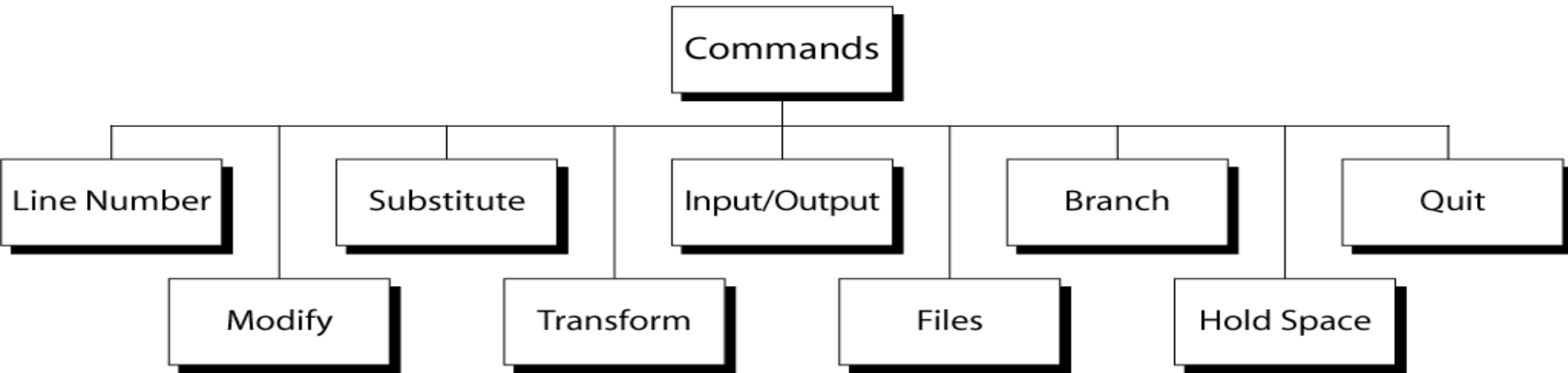
4,9 (Matches all lines 4 through 9)

2,/ ^A/command (starts matching from line 2 to a line that starts with A)

/ ^A/,15 (starts matching from a line that begins with A to 15)

/ ^A/,/ \$B/command (starts matching from a line that begins with A to a line that ends with B)

Commands



Line Number

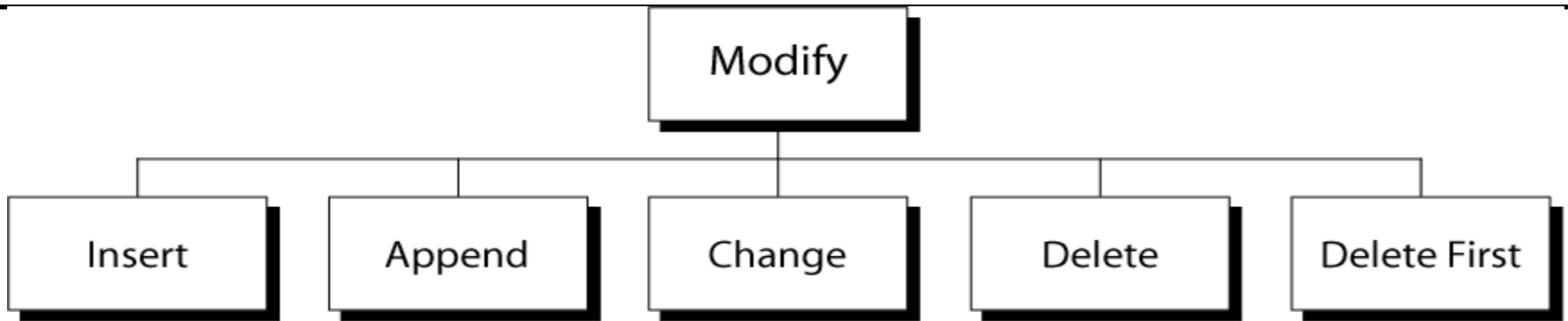
line number command (=) writes the current line number before each matched/output line

Example:

```
$ sed '=' twister.dat
```

This command print the file twister.dat with line number

Commands



Insert Command: I

- adds one or more lines directly to the output before the address:
- inserted “text” never appears in sed’s pattern space
- cannot be used with a range address; can only be used with the single-line and set-of-lines address types

Syntax:

- [address] i\
text \

Commands

Append Command: a

- adds one or more lines directly to the output after the address:
- Similar to the insert command (i), append cannot be used with a range address.
- Appended `—text` does not appear in sed's pattern space.

Syntax:

```
[address] a\  
      text
```

EX:

Script.sed

#script to append the text at the end of the file

\$a\
Is it difficult? Try again!

Addresses

Change Command: c

- replaces the lines matching the address with x:new text
- accepts four address types:
- single-line, set-of-line, range, and nested addresses.

Syntax:

- **[address1[,address2]] c\
text**

Ex: \$sed -f script.sed twister.dat

Script.sed

5c\

Red Blood Blue Blood\

Deadly Dinosaurs Danced Dizzly

o/p:

Bad Black Bread

Fresh Fried Fish

Commands

Ex: to delete all the lines that starts with a capital letter either B or I

```
$sed '/^[BI]/d' twister.dat
```

o/p:

Fresh Fried Fish

Six Thick Thistle Sticks

Short Socks with Spots

Commands

Substitute command:

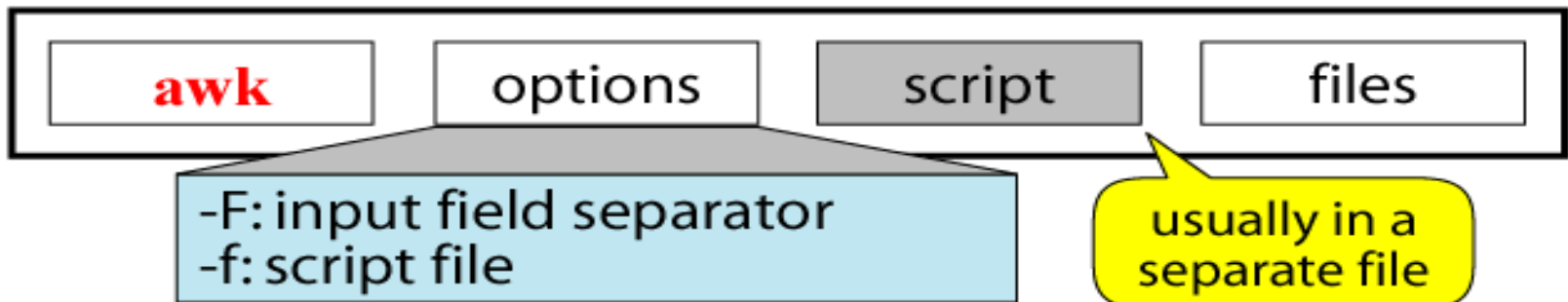
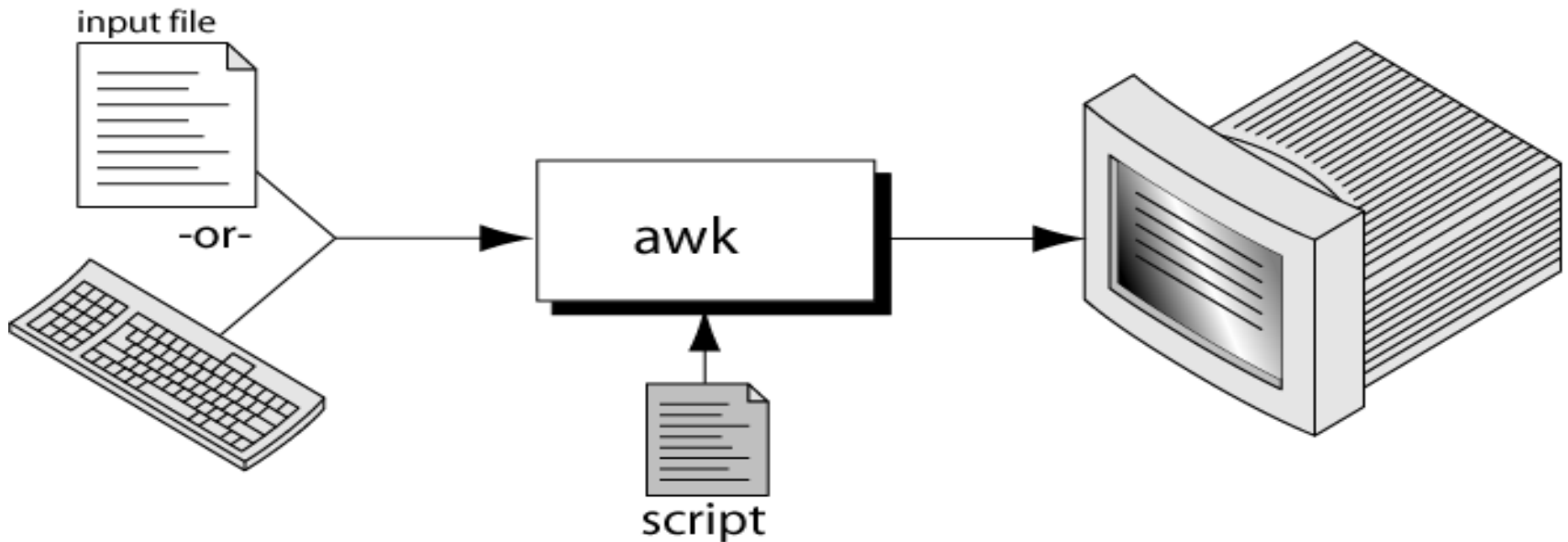
The substitute command, replaces the text matching the search pattern with a replacement string.

Syntax:

- *[address(es)]s/pattern/replacement/[flags]*
 - *pattern* - search pattern
 - *replacement* - replacement string for pattern
 - *flags* - optionally any of the following
 - **n** it replaces only the specified occurrence of *pattern*
 - Here n is an integer
 - **g** it replaces all occurrences of a *pattern* in pattern space
 - By default it replaces the first occurrences of a pattern
 - **p** print contents of pattern space
 - **W** it writes a file

Awk

The awk utility, which takes its name from the initials of its authors (Aho, Weinberger, and Kernighan), is a powerful programming language.



Execution

In addition to input data, awk also requires one or more instructions that provide editing instructions.

A few instructions can be entered at the command line from the keyboard.

Most of the time, the instructions are placed in a file known as an awk script.

Syntax

`$awk options awk scripts file(s)`

Options

`-f`: it indicates that script is in a file which immediately follows this option.

`-F`: this option specifies the input field separator. By default, it is space or a tab.

Ex:

```
$awk -F "\t" -f firstscript.awk info.dat
```

Fields and Records

The awk utility views a file as a collection of fields and records.

A field is a unit of data that has informational content.

Each field of information is separated by one or more whitespace characters or other separator.

Each line is a record. A record is a collection of fields.

When a file is organized into records, we call it a data file, as contrasted with a text file.

awk also can handle a text file.

A Data File

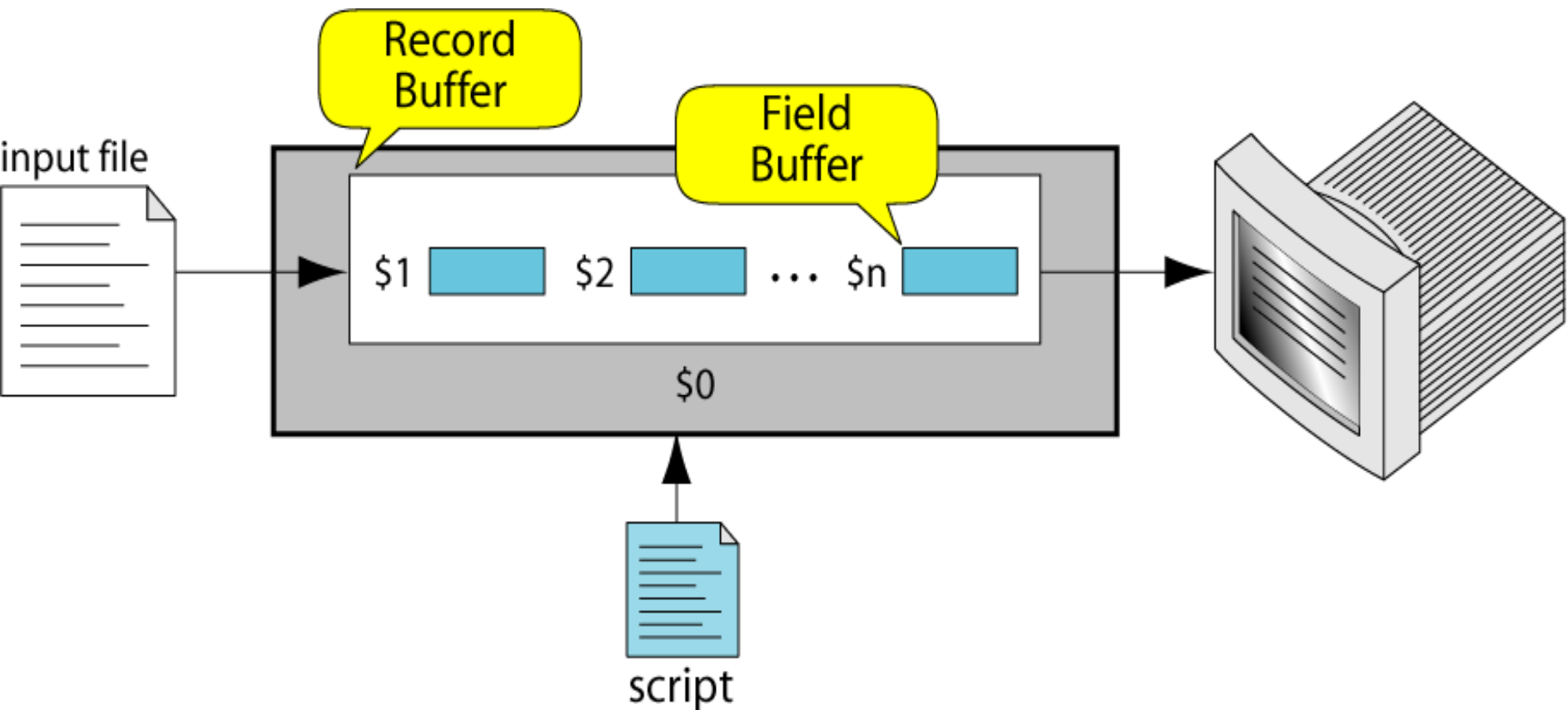
	Field 1 (First_Name)	Field 2 (Last_Name)	Field 3 (Pay_Rate)	Field 4 (Hours)
Record 2	Susan	White	6.00	23
	Mark	Eagle	6.25	40
Record 4	Tuan	Nguyen	7.89	44
	Dan	Black	7.23	40
	Amanda	Trapp	6.95	40
	Brian	Devaux	7.95	0
	Chris	Walljasper	6.89	32
	Mary	Lamb	8.22	40
	Jackie	Kammaoto	7.59	40
Record 10	Nicky	Barber	6.35	40

A file with 10 records, each with four fields

Field Buffers

There are as many field buffers available as there are fields in the current record of the input file.

Each buffer has a name, which is the dollar sign \$ followed by the field number in the current record.



Record Buffer

There is only one record buffer available. Its name is \$0.

It holds the whole record.

System Variables

FS	Input field separator
RS	Input record separator
OFS	Output field separator
ORS	Output record separator
NF	Number of nonempty fields in a record
NR	Number of records read from all files
FNR	Number of records in current file
FILENAME	Name of the current file

System Variables

Examples :

% cat emps

Tom Jones 4424 5/12/66 543354

Mary Adams 5346 11/4/63 28765

Sally Chang 1654 7/22/54 650000

Billy Black 1683 9/23/44 336500

% awk '{print NR, \$0}' emps

1 Tom Jones 4424 5/12/66 543354

2 Mary Adams 5346 11/4/63 28765

3 Sally Chang 1654 7/22/54 650000

4 Billy Black 1683 9/23/44 336500

awk Script Design

BEGIN {Begin's Actions}

Preprocessing

Pattern {Action}

Pattern {Action}

Pattern {Action}

Body

END {End's Actions}

Postprocessing

awk Script

- BEGIN: pre-processing
 - performs processing that must be completed before the file processing starts (i.e., before awk starts reading records from the input file)
 - useful for initialization tasks such as to initialize variables and to create report headings
- BODY: Processing
 - contains main processing logic to be applied to input records
 - like a loop that processes input data one record at a time:
 - if a file contains 100 records, the body will be executed 100 times, one for each record

awk Script

- END: post-processing
 - contains logic to be executed after all input data have been processed
 - logic such as printing report grand total should be performed in this part of the script

Operations

- The awk utility reads an input file, line by line and applies all the instructions in awk script to each input line. It does not print the lines unless the print action is specified .consider a records .dat with five fields i.e,s.no,name,test1,test2,test3 and four records.

1	A	75	81	89
2	B	86	94	82
3	C	90	71	96
4	D	82	70	85

```
$awk -f total.awk records.dat
```

```
Total.awk
```

```
#initialization section
```

```
BEGIN {print "name\t total"}
```


Operations

#body section

```
{total=$3+$4+$5}
```

```
{print $2+"\" total}
```

#END of job

```
END {print "Best Result"}
```

Output:

Name	total
------	-------

A	245
---	-----

B	262
---	-----

C	257
---	-----

D	237
---	-----

Best Result

Pattern/Action

```
pattern {statement}
```

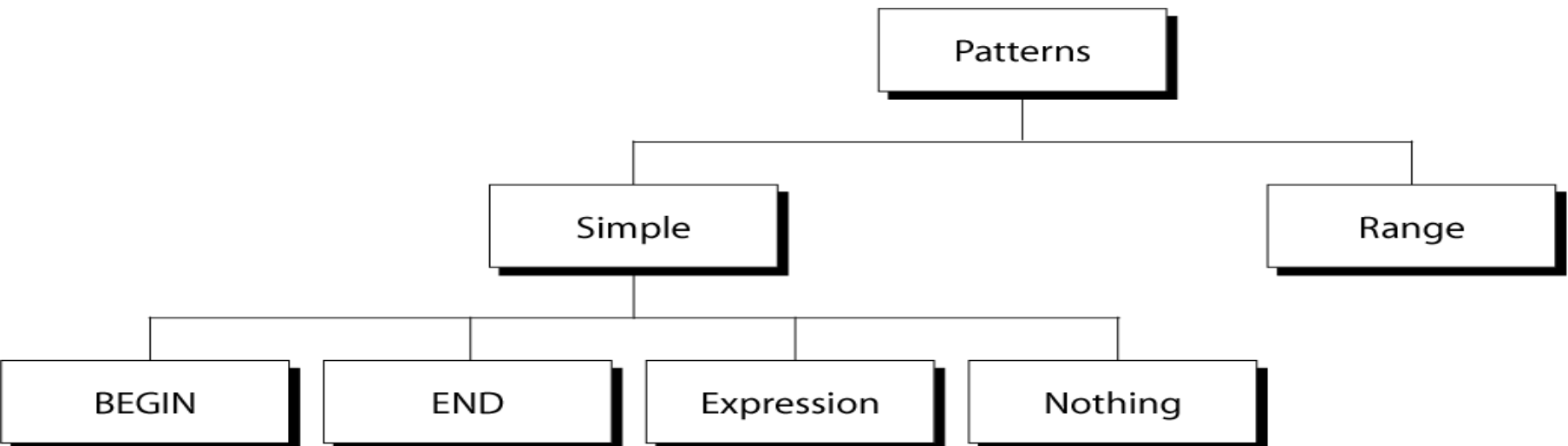
(a) One Statement Action

```
pattern {statement1; statement2; statement3}
```

(b) Multiple Statements Separated by Semicolons

```
pattern  
{  
    statement1  
    statement2  
    statement3  
}
```

(c) Multiple Statements Separated by Newlines



Expression Pattern types

- match
 - entire input record
 - regular expression enclosed by ‘/’s
 - explicit pattern-matching expressions
 - ~ (match), !~ (not match)
- expression operators
 - arithmetic
 - relational
 - logical

Example: match input record

```
% cat employees2
```

```
Tom Jones:4424:5/12/66:543354
```

```
Mary Adams:5346:11/4/63:28765
```

```
Sally Chang:1654:7/22/54:650000
```

```
Billy Black:1683:9/23/44:336500
```

```
% awk -F: '/00$/' employees2
```

```
Sally Chang:1654:7/22/54:650000
```

```
Billy Black:1683:9/23/44:336500
```

Arithmetic Operators

<u>Operator</u>	<u>Meaning</u>	<u>Example</u>
+	Add	$x + y$
-	Subtract	$x - y$
*	Multiply	$x * y$
/	Divide	x / y
%	Modulus	$x \% y$
^	Exponential	$x \wedge y$

Example:

```
% awk '$3 * $4 > 500 {print $0}' file
```

Relational Operators

<u>Operator</u>	<u>Meaning</u>	<u>Example</u>
<	Less than	$x < y$
< =	Less than or equal	$x < = y$
==	Equal to	$x == y$
!=	Not equal to	$x != y$
>	Greater than	$x > y$
> =	Greater than or equal to	$x > = y$
~	Matched by reg exp	$x \sim /y/$
!~	Not matched by req exp	$x !\sim /y/$

Logical Operators

<u>Operator</u>	<u>Meaning</u>	<u>Example</u>
&&	Logical AND	a && b
	Logical OR	a b
!	NOT	! a

Examples:

```
% awk '($2 > 5) && ($2 <= 15)
```

```
{print $0}' file
```

```
% awk '$3 == 100 || $4 > 50' file
```

Logical Operators

<u>Operator</u>	<u>Meaning</u>	<u>Example</u>
&&	Logical AND	a && b
	Logical OR	a b
!	NOT	! a

Examples:

```
% awk '($2 > 5) && ($2 <= 15)
```

```
{print $0}' file
```

```
% awk '$3 == 100 || $4 > 50' file
```


Range Patterns

Matches ranges of consecutive input lines

Syntax:

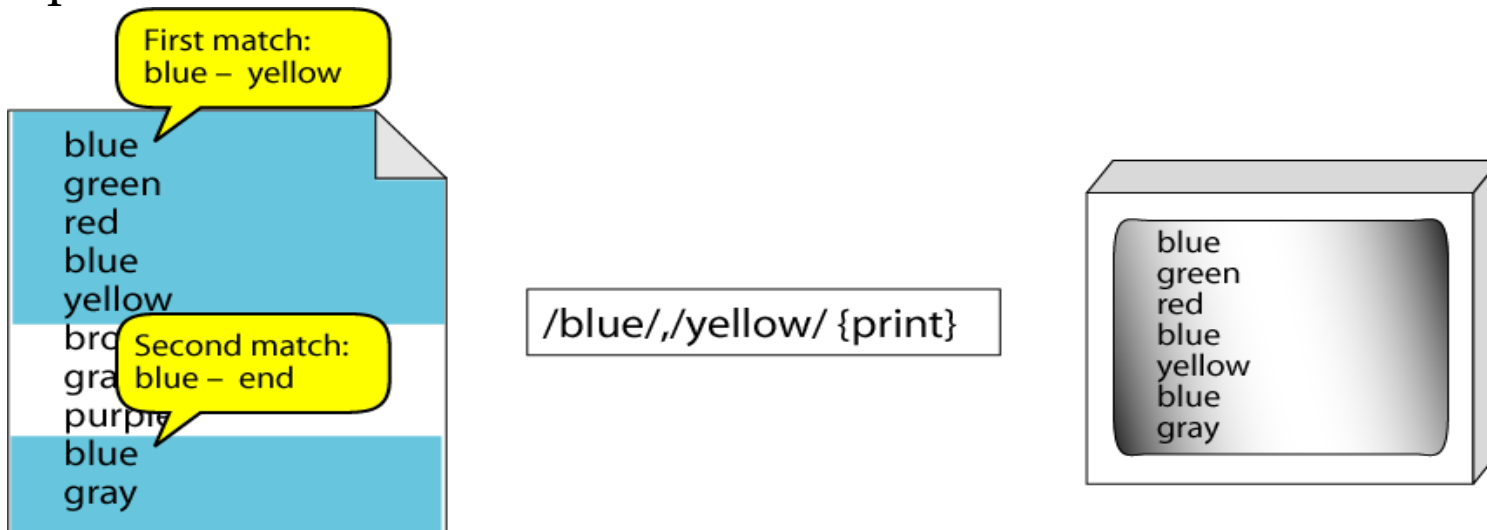
pattern1 , pattern2 {action}

pattern can be any simple pattern

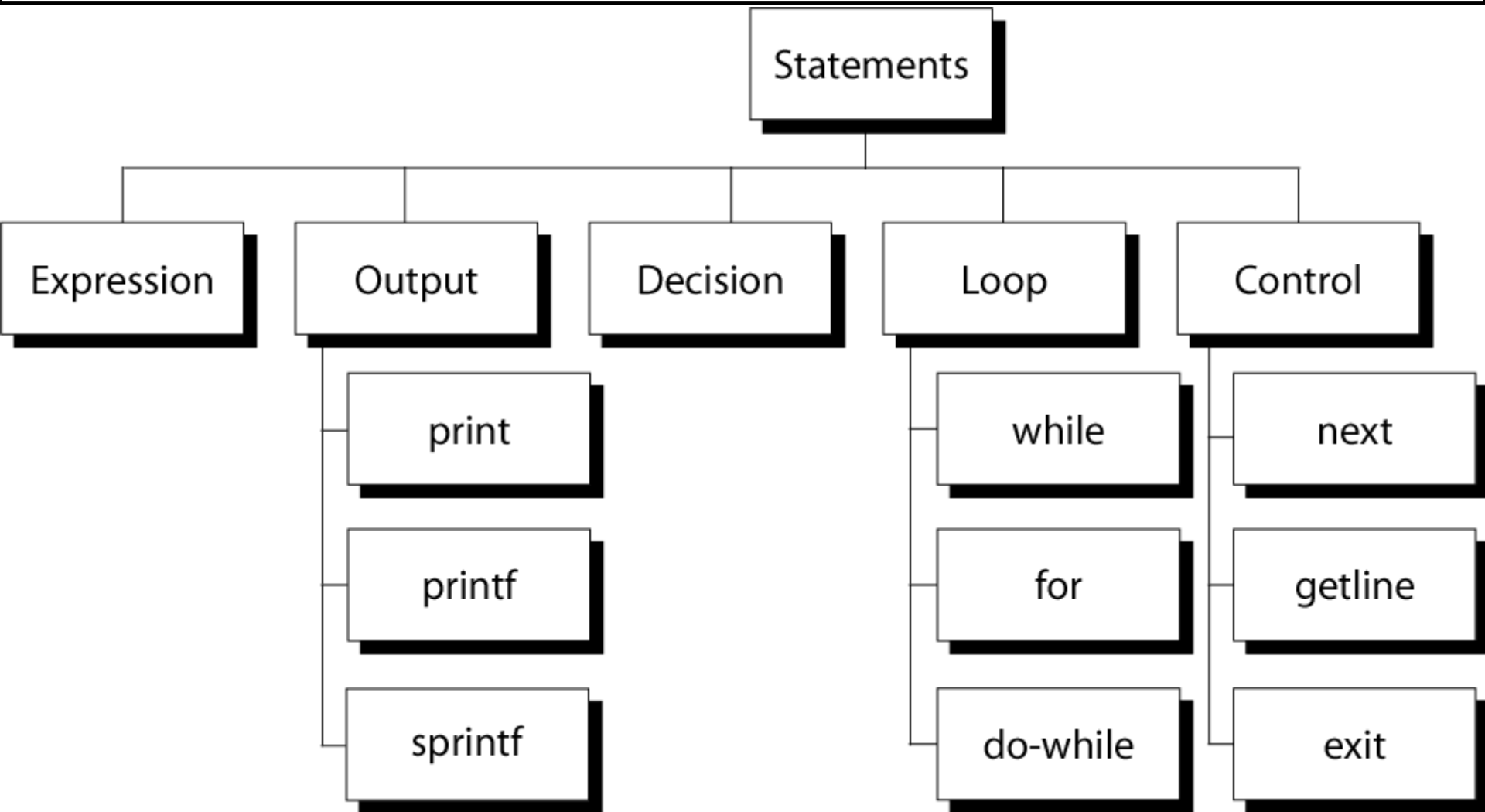
pattern1 turns action on

pattern2 turns action off

Example:



awk Actions



awk expressions

Expression is evaluated and returns value

- consists of any combination of numeric and string constants, variables, operators, functions, and regular expressions

Can involve variables

- As part of expression evaluation
- As target of assignment

awk variables

- A user can define any number of variables within an awk script
- The variables can be numbers, strings, or arrays
- Variable names start with a letter, followed by letters, digits, and underscore
- Variables come into existence the first time they are referenced; therefore, they do not need to be declared before use
- All variables are initially created as strings and initialized to a null string ""

Format:

variable = expression

Examples:

```
% awk '$1 ~ /Tom/  
    {wage = $3 * $4; print wage}'  
filename  
  
% awk '$4 == "CA"  
    {$4 = "California"; print $0}'  
filename
```

awk assignment operators

= assign result of right-hand-side expression to
left-hand-side variable

++ Add 1 to variable

-- Subtract 1 from variable

+= Assign result of addition

-= Assign result of subtraction

*= Assign result of multiplication

/= Assign result of division

%= Assign result of modulo

^= Assign result of exponentiation

Output Statements

print

print easy and simple output

printf

print formatted (similar to C printf)

sprintf

format string (similar to C sprintf)

Awk control structures

- Conditional

- if-else

- Repetition

- for

- with counter

- with array index

- while

- do-while

- also: break, continue

if Statement

Syntax:

```
if (conditional expression)
```

```
    statement-1
```

```
else
```

```
    statement-2
```

Example:

```
if ( NR < 3 )
```

```
    print $2
```

```
else
```

```
    print $3
```


for Loop

Syntax:

```
for (initialization; limit-test; update)
    statement
```

Example:

```
for (i = 1; i <= NR; i++)
{
    total += $i
    count++
}
```

- For loop for arrays

Syntax:

```
for (var in array)
    statement
```

Example:

```
for (x in deptSales)
{
    print x, deptSales[x] }
```

while Loop

Syntax:

```
while (logical expression)  
    statement
```

Example:

```
i = 1  
while (i <= NF)  
{  
    print i, $i  
    i++  
}
```

do-while Loop

Syntax:

do

statement

while (condition)

- statement is executed at least once, even if condition is false at the beginning

Example:

```
i = 1
```

```
do {
```

```
    print $0
```

```
    i++
```

```
} while (i <= 10)
```